



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Rodrigo Alejandro Barbosa Vidal

Fecha:
24/05/26

Tarea 4 Análisis del Ciclo de Vida del Software (SDLC)

Sistema seleccionado: Airbnb

Introducción

Los software requiere un proceso organizado que permitan crear sistemas seguros y funcionales, para esto, las empresas requieren hacer lo que se le conoce como ciclo de vida del desarrollo de software, el cual se divide en varias etapas que ayudan a planear, diseñar, desarrollar e implementar sistemas de manera rentable, en esta tarea se analizará cómo se aplican las etapas en la plataforma Airbnb, una aplicación utilizada mundialmente para reservar alojamientos y experiencias, explicaremos cómo funciona el sistema, sus requerimientos, diseño, pruebas, implementación y mantenimiento, demostrando la importancia de cada etapa dentro del desarrollo de software.

Descripción del Sistema

Nombre del sistema:

Airbnb

¿Qué problema resuelve?

la necesidad de encontrar hospedaje de manera rápida y accesible, permitiendo que las personas renten casa, departamento o habitaciones desde cualquier parte del mundo.

Tipo de usuarios:

viajeros, administradores de la plataforma, equipo de soporte técnico

Funcionalidades principales:

Registro e inicio de sesión, Búsqueda de alojamientos, Reservación de hospedajes, Sistema de pagos en línea, Calificaciones y comentarios, Chat entre huésped y anfitrión, gestión de propiedades.

Importancia del sistema:

Facilita el acceso a hospedajes a nivel mundial, conectando viajeros mediante una plataforma digital fácil de utilizar.

Etapas de Planeación

Objetivos del sistema:

Permitir reservar alojamientos en línea, facilitar la comunicación entre usuarios, Garantizar pagos seguros, ofrecer disponibilidad del sistema las veinticuatro horas.

Alcance general:

El sistema funcionará en navegadores web, aplicaciones móviles, tablets, diferentes sistemas operativos.

Recursos necesarios

Recursos humanos:

Ingenieros de software, Diseñadores, Especialistas en bases de datos, Expertos en ciberseguridad, Administradores de servidores.

Recursos tecnológicos:

Bases de datos, APIs de pago, Servidores web, Sistemas de monitoreo.

Riesgos principales:

Robo de información, fallas en pagos, saturación de usuarios, errores en reservaciones, problemas de seguridad

Tiempo estimado:

El desarrollo de un sistema similar podría tomar 1 o 2 años debido a su evolución, crecimiento constante.

Costos aproximados

El costo de desarrollar una aplicación Airbnb varía ampliamente según la complejidad, funcionalidades, diseño y equipo de desarrollo, pero generalmente oscila entre \$30,000 y más de \$150,000 para el desarrollo inicial.

Aplicación básica : Entre \$30,000 y \$70,000, incluyendo funciones esenciales como listado de propiedades, búsqueda, reservas, pagos y perfiles de usuario.

Aplicación avanzada: Puede superar los \$100,000–\$150,000, especialmente si incluye características personalizadas, algoritmos de recomendación, integraciones complejas con APIs, diseño premium y desarrollo multiplataforma (iOS, Android, web).

Factores clave: El costo depende del equipo (freelancers, agencia local o en Europa del Este), la tecnología usada, el diseño personalizado y el tiempo de desarrollo, que puede extenderse entre 3 y 9 meses o más, hay que considerar costos recurrentes como mantenimiento (entre \$2,000 y \$10,000 mensuales), hosting, actualizaciones y seguridad.

Análisis de Requerimientos

Requerimientos funcionales

1. Los usuarios podrán crear una cuenta dentro de la plataforma.

2. La aplicación ofrecerá herramientas para buscar alojamientos disponibles.
3. Será posible realizar reservaciones de hospedaje directamente desde el sistema.
4. La plataforma gestionará pagos electrónicos de manera segura.
5. Los huéspedes y anfitriones tendrán acceso a un sistema de mensajería interna.
6. El sistema mostrará opiniones, valoraciones y comentarios de los usuarios.
7. Los anfitriones podrán administrar y actualizar la información de sus propiedades.

Requerimientos no funcionales

1. El sistema debe estar disponible 24/7.
2. Debe soportar miles de usuarios simultáneos.
3. La información debe estar cifrada.
4. La plataforma debe tener tiempos de respuesta rápidos.
5. Debe ser compatible con dispositivos móviles.
6. La interfaz debe ser intuitiva.
7. Debe contar con respaldo automático de información.

Diseño del Sistema

Arquitectura general:

El sistema podría utilizar una arquitectura de microservicios para separar funcionalidades como pagos, reservaciones, usuarios y mensajería.

Base de datos, Base SQL para usuarios y pagos, Base NoSQL para mensajes y búsquedas rápidas.

Tecnologías posibles:

Frontend: React, HTML, CSS y JavaScript.

Backend: Node.js, Java o Python.

Base de datos: MySQL y MongoDB.

Nube: AWS o Google Cloud.

Interfaces:

Plataforma web, Aplicación móvil, Panel administrativo.

Seguridad:

Inicio de sesión seguro, cifrado HTTPS, verificación de usuarios, protección contra ataques cibernéticos

APIs o servicios externos:

API de pago, Servicios de mapas, API de geolocalización, Servicios de autenticación.

Desarrollo

Roles del equipo:

Project Manager, Desarrolladores frontend, Desarrolladores backend, Diseñadores, testers, especialistas en seguridad.

Tecnologías de desarrollo:

Git y GitHub, Frameworks frontend y backend.

Organización del trabajo:

El proyecto se desarrollará utilizando metodología Scrum mediante sprints y reuniones periódicas.

Control de versiones:

Utilizaremos Git para gestionar cambios y colaboración del equipo.

Posibles dificultades:

Escalabilidad del sistema, integración de servicios externos, seguridad de pagos, compatibilidad entre dispositivos

Pruebas

Pruebas funcionales:

Verificación registro de usuarios, Inicio de sesión, Reservaciones, Pagos, Mensajería.

Pruebas de seguridad:

Protección contra robo de datos, Pruebas de vulnerabilidades, Seguridad en pagos

Pruebas de rendimiento:

Carga de usuarios simultáneos, Tiempo de respuesta, Rendimiento del servidor

Pruebas de integración:

Verificación de conexión entre base de datos, API externas, Sistema de pagos

Pruebas de usuario:

Usuarios reales evaluarían: Facilidad de navegación, Diseño visual, Experiencia de uso

Implementación

Infraestructura:

El sistema se desplegará en servidores distribuidos globalmente.

Hosting o nube:

Uso de plataformas cloud como AWS e Google Cloud.

Publicación:

La aplicación se publicaría en app store, google play, navegadores web.

Capacitación de usuarios:

Tendrá tutoriales, apartado de preguntas frecuentes y centro de ayuda.

Migración de información:

Los datos se transfieren de forma segura y sin pérdidas.

Mantenimiento

Corrección de errores:

Resolver problemas reportados por usuarios.

Actualizaciones:

Mejorar funciones y apariencia del sistema en base a feedback de los usuarios.

Nuevas funcionalidades:

Mejoras de búsqueda, Nuevos métodos de pago, Mejoras en mensajería.

Seguridad:

Actualizar mecanismos de protección continuamente.

Optimización:

Mejorar velocidad, estabilidad y rendimiento.

Análisis y Reflexión

1. ¿Qué etapa del SDLC consideras más importante y por qué?

La más importante es el análisis de requerimientos y porque permite identificar correctamente las necesidades del sistema antes de comenzar el desarrollo.

2. ¿Qué riesgos existirían si se omite una etapa?

Podrían presentarse errores, atrasos, fallas de seguridad y un sistema que no cumpla las necesidades de los usuarios.

3. ¿Por qué el mantenimiento es crítico en sistemas modernos?

porque requieren actualizaciones constantes, corrección de errores y protección frente a amenazas de seguridad, ya que es muy frecuente que se lleguen a encontrar fallas en ciertas apps

4. ¿Cómo ayuda el SDLC a mejorar calidad y organización?

organiza el trabajo del equipo, reduce riesgos y permite desarrollar software más seguro, eficiente y de mejor calidad.

Conclusión

El SDLC es fundamental para desarrollar sistemas modernos de manera organizada y eficiente, debido al análisis realizado sobre Airbnb, se puede observar cómo cada etapa contribuye al correcto funcionamiento de la plataforma, el ciclo de vida del software ayuda a mejorar la calidad del sistema, controlar riesgos y facilitar el mantenimiento, siendo una herramienta esencial en el desarrollo.